

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO PRÁTICA DE CONEXÕES SEGURAS

Faculdade de Tecnologia SENAI Félix Guisard Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Projeto Semestral — Parte 1/3 | Data de entrega: Maio de 2026

Introdução

Este relatório documenta as **Provas de Conceito (PoCs)** desenvolvidas para o projeto *Conexões Seguras*, cobrindo três mecanismos fundamentais de segurança em redes corporativas modernas. O objetivo é demonstrar na prática tanto as vulnerabilidades de protocolos e configurações legadas quanto a eficácia das arquiteturas modernas de proteção.

Ambiente de laboratório: Instância Ubuntu Server 22.04 LTS provisionada na infraestrutura AWS (Cloud9/EC2, tipo t3.micro), garantindo um ambiente isolado e reproduzível para todos os experimentos.

Módulo 1: Sniffing e Interceptação (Camada 2 e 3)

Ferramenta Usada: iproute2 (comando ip), tcpdump e curl

Passo a Passo

1. Instalação das dependências no ambiente via `sudo apt update && sudo apt install tcpdump curl -y`
2. Identificação da interface de rede ativa com `ip link show` (interface `ens5` em EC2)
3. Habilitação do modo promíscuo no Controlador de Interface de Rede (NIC): `sudo ip link set dev ens5 promisc on`
4. Inicialização do sniffer para captura de tráfego HTTP na porta 80, com payload em ASCII: `sudo tcpdump -i ens5 -nn -A -s 0 port 80`
5. Em terminal secundário, injeção de tráfego não criptografado: `curl http://neverssl.com`
6. Restauração da interface ao estado original: `sudo ip link set dev ens5 promisc off`

Resultado

O tcpdump exibiu imediatamente o fluxo de dados em texto legível. A captura comprovou a interceptação do cabeçalho da requisição (GET / HTTP/1.1), do campo Host: e do corpo HTML completo da resposta do servidor de destino. (*Evidências: evidências/comandom1.png e evidencias/resultadom1.png*)

Explicação Técnica do Resultado

Modo Promíscuo e o comportamento da NIC: Em operação padrão, o controlador de interface de rede (NIC) implementa um filtro de endereços MAC no nível do driver: apenas quadros Ethernet cujo MAC de destino coincide com o da própria interface (ou com o MAC de broadcast/multicast) são encaminhados à pilha de rede do sistema operacional. Todos os demais são descartados silenciosamente pelo hardware.

O **modo promíscuo** desabilita esse filtro, forçando a NIC a encaminhar **todos os quadros interceptados no segmento de rede** para a CPU, independentemente do endereço MAC de destino. Como o protocolo HTTP não opera sobre uma camada de transporte segura (TLS), os dados trafegam em **Texto Claro (Plain Text)**, expondo cabeçalhos, credenciais e conteúdo a qualquer host no mesmo segmento que esteja capturando tráfego. Isso comprova a vulnerabilidade crítica de comunicações locais não encriptadas frente a ataques de interceptação passiva na Camada 2 (Data Link).

Módulo 2: Criptografia e Handshake (A Base do TLS)

Ferramenta Usada: OpenSSL

Passo a Passo

1. **Geração Assimétrica (Servidor):** Criação de chave privada RSA de 2048 bits via openssl genpkey e extração da chave pública via openssl rsa
2. **Geração Simétrica (Cliente):** Criação de chave de sessão AES-256 de 32 bytes via openssl rand
3. **Simulação do Handshake:** O cliente utilizou a Chave Pública RSA do servidor para criptografar a chave simétrica de sessão, gerando encrypted_symmetric.bin
4. **Transferência Segura:** Arquivo de dados sensíveis criptografado com AES-256-CBC e a chave simétrica, gerando secret.enc
5. **Recepção (Servidor):** Uso da Chave Privada RSA para revelar a chave de sessão e, com ela, descriptografar o payload intacto

Resultado

O servidor reconstruiu e exibiu a mensagem original: *"Dados corporativos extremamente confidenciais - Módulo 2"*. Durante a transferência, os arquivos intermediários (encrypted_symmetric.bin e secret.enc) permaneceram em **formato binário ilegível** — inúteis para qualquer interceptador sem acesso à chave privada. (Evidência: evidencias/comandosm2.png)

Explicação Técnica do Resultado

Por que o HTTPS não usa apenas criptografia assimétrica (RSA/ECC)? A criptografia assimétrica fundamenta-se em operações matemáticas computacionalmente custosas — no RSA, a fatoração de números primos de centenas de dígitos. Se todo o tráfego HTTPS (imagens, vídeos, APIs, streaming) fosse criptografado via RSA, o consumo de CPU em servidores e clientes entraria em colapso, tornando a latência inaceitável para qualquer aplicação real.

A **arquitetura híbrida** resolve isso usando cada algoritmo para sua força específica: a **criptografia assimétrica (RSA/ECC)** é utilizada exclusivamente na fase de Handshake, com o único propósito de trocar com segurança a chave de sessão por um canal inseguro. Uma vez que ambos os lados possuem a chave de sessão compartilhada, toda a comunicação subsequente utiliza **criptografia simétrica (AES-256)**, que possui suporte de aceleração direto no hardware dos processadores modernos (instruções AES-NI), garantindo transferência de dados em altíssima velocidade com overhead computacional mínimo.

Módulo 3: Segurança na Camada de Aplicação (Desenvolvimento)

Ferramenta Usada: Node.js, framework Express.js e curl

Passo a Passo

1. Inicialização de projeto Node.js com `npm init -y` e instalação do express
2. Criação do servidor web `server.js` escutando na porta 3000
3. Implementação de dois endpoints com comportamentos opostos:
 - `/login-inseguro`: injeta SessionID sem diretivas de proteção (`httpOnly: false`)
 - `/login-seguro`: injeta SessionID com diretivas `{ secure: true, httpOnly: true }`
4. Inspeção dos cabeçalhos de resposta via `curl -v` para ambas as rotas

Resultado

Rota insegura: `< Set-Cookie: SessionID=abc123vulneravel; Path=/`

O cookie é transmitido sem restrição alguma — exposto a sniffing de rede e scripts JavaScript.

Rota segura: `< Set-Cookie: SessionID=xyz987protegido; Path=/; HttpOnly; Secure`

O cookie possui ambas as flags de segurança ativadas, bloqueando os vetores de ataque mais comuns. (*Evidências: evidencias/cookiesm3.png, evidencias/serverm3.png*)

Explicação Técnica do Resultado

O que é o Cookie SessionID e quais são as preocupações para o desenvolvedor? O protocolo HTTP é *stateless* — o servidor não mantém estado entre requisições. O SessionID funciona como um "crachá digital": gerado pelo servidor após o login e armazenado no navegador, é enviado automaticamente em cada requisição subsequente para autenticar o usuário.

O risco crítico para o desenvolvedor é o **Session Hijacking**: se o cookie trafegar via HTTP puro (sem TLS), um atacante na mesma rede utilizando modo promíscuo (como demonstrado no Módulo 1) pode interceptar o SessionID em trânsito e injetá-lo no próprio navegador, assumindo controle total da conta da vítima — sem necessidade de senha. Um segundo vetor é o **Cross-Site Scripting (XSS)**: um script malicioso injetado na página pode acessar `document.cookie` e exfiltrar o token.

O papel do HTTPS e das flags de segurança: A flag Secure instrui o navegador a nunca transmitir o cookie em conexões HTTP — o cookie só é incluído nas requisições quando um túnel TLS (Módulo 2) está estabelecido, neutralizando a interceptação de rede. A flag HttpOnly remove o cookie do acesso JavaScript, bloqueando o vetor XSS. A combinação de HTTPS + Secure + HttpOnly constitui uma **defesa em profundidade**: protege o canal de rede, previne vazamentos por HTTP acidental e elimina o roubo via scripts.

Conclusão

As três Provas de Conceito desenvolvidas demonstram uma visão integrada da segurança em camadas (*Defense in Depth*):

Módulo	Camada OSI	Ameaça Demonstrada	Mitigação
1 — Sniffing	Camada 2/3	Interceptação passiva de tráfego em texto claro	Uso obrigatório de TLS/HTTPS
2 — TLS	Camada 4/5	Custo computacional inviável de RSA puro	Criptografia Híbrida (RSA + AES)
3 — Sessões	Camada 7	Session Hijacking via HTTP e XSS	HTTPS + flags Secure e HttpOnly

Os experimentos evidenciam que a segurança não é responsabilidade de uma única tecnologia, mas o resultado da aplicação correta de múltiplos mecanismos complementares em cada camada da pilha de comunicação. A transição de HTTP para HTTPS, aliada à gestão adequada de sessões na camada de aplicação, forma a base essencial de segurança exigida em ambientes de produção modernos.